

Day05 Part E 学习文档 v1.0 : Tool Executor (Execution Runtime)

本文是《从零实现 Agent Runtime》学习阶段的 Day05 Part E 正式学习文档。

Day05 Part A 建立 Tool Calling 基础模型，Part B 解释 LLM 如何做 Tool Decision，Part C 说明 Tool Schema 如何定义单个 Tool 的行动契约，Part D 说明 Tool Registry 如何管理 Runtime 的能力空间。Part E 继续回答执行层问题：当 LLM 已经产生 Tool Call Intent 后，Runtime 如何把这个意图变成一次安全、可靠、可观测的真实世界动作？

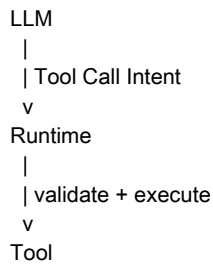
本节定位

Day05 的主题是 Tool Calling，也就是 Agent Runtime 的 Execution Engine。

Part A 回答：

Tool Calling 是什么？

核心是：



Part B 回答：

LLM 为什么会决定调用 Tool？

核心是：

Tool Decision = Goal-driven Action Selection

Part C 回答：

Runtime 如何描述一个 Tool，让 LLM 能正确理解、选择、填参和解释结果？

核心是：

Tool Schema = Action Contract

Part D 回答：

Runtime 如何管理大量 Tool，并决定当前这一轮应该给 LLM 看哪些 Tool？

核心是：

Tool Registry = Capability Management Center

Part E 继续回答：

LLM 已经选择了 Tool，也生成了参数，那么谁真正执行？谁校验参数？谁判断权限？谁处理失败？谁记录日志？谁控制超时？谁把结果反馈给 Runtime State？

答案是：

Tool Executor = Execution Runtime

本节的核心结论是：

Tool Executor 不是 tool.execute() 的一层薄封装，而是 Agent Runtime 中负责将 LLM 产生的 Tool Call Intent 转换为真实外部动作，并管理整个执行生命周期的执行内核。它是 LLM 决策层与真实世界之间的安全边界、可靠性边界和可观测性边界。

目录

与 Part D 的联系

为什么需要 Tool Executor

Tool Executor 是什么

为什么 LLM 不能直接执行 Tool

Tool Executor 在 Runtime Loop 中的位置

Tool Call Intent 生命周期

Intent、Registry、Router 与 Executor 的关系

Tool Schema 是 LLM 行动空间的边界

Tool Executor 的职责边界

Tool Execution Context

Input Validation

Permission Check 与 Human Approval 的边界

Reliability：Timeout、Retry、Cancellation

Idempotency

Error Observation

Tool Result Processing

Streaming 与 Long-running Tool

Executor 与 Workflow 的边界

Static Tool 与 Dynamic Tool

MCP 与 Dynamic Tool Registry

前端动态物料体系类比 Agent Tool Platform

mini-agent-runtime 中的最小数据模型

mini-agent-runtime 最小实现草图

工业术语映射

面试视角

下一节学习计划

写书 TODO

写书素材

本 Part 核心认知升级

本章思考题

前置问题回收

与 Part D 的联系

Part D 讨论的是 Tool Registry。

它关心：

- Runtime 有哪些能力？
- 这些能力来自哪里？
- 当前任务应该暴露哪些能力给 LLM？
- 这些能力如何按权限、上下文、风险和相关性筛选？

所以 Part D 的核心链路是：

```
All Tools
|
v
Tool Registry
|
v
Tool Router
|
v
Current Available Tools
|
v
Context Builder
|
v
LLM-visible Tool Schemas
```

Part E 接在这里。

当 LLM 在当前可见 Tool Schema 中选择一个 Tool，并输出：

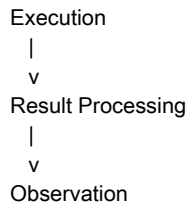
```
{
  "name": "query_order",
  "arguments": {
    "order_id": "ORD-10001"
  }
}
```

Part D 的工作已经完成：

- Registry 管理能力
- Router 缩小能力空间
- Context Builder 暴露能力描述
- LLM 选择能力

Part E 的工作才刚开始：

```
Tool Call Intent
|
v
Executor
|
v
Validation
|
v
Permission
|
v
```



因此：

Tool Registry 负责管理能力。
Tool Executor 负责执行能力。

为什么需要 Tool Executor

最简单的 Demo 里，很多人会这样写：

```
const result = await tools[toolCall.name](toolCall.arguments);
```

当 Tool 很少、风险很低、执行很快、没有权限和审计要求时，这可以跑通。

但真实 Agent Runtime 中，Tool Executor 必须回答很多更工程化的问题：

- 这个 Tool 是否存在？
- 这个 Tool 当前是否可用？
- 模型生成的参数是否符合 schema？
- 参数是否满足业务约束？
- 当前用户是否有权限？
- 是否需要人工审批？
- 是否需要超时控制？
- 失败是否可以重试？
- 如果重试，是否会造成重复扣款或重复退款？
- Tool 返回的大结果是否能直接进入 Context？
- 失败结果是终止任务，还是作为 Observation 继续进入下一轮？
- 执行过程如何打日志、追踪和审计？
- 长时间任务如何流式回报进展？

这些都不是 LLM 应该负责的。

它们属于：

Runtime Managed Execution

这就是 Tool Executor 存在的原因。

Tool Executor 是什么

一个简洁定义：

Tool Executor 是 Agent Runtime 中负责接收 Tool Call Intent、查找 Tool Implementation、验证输入、检查权限、管理执行生命周期、标准化结果，并将结果写回 Runtime State 的执行组件。

它不是单纯函数调用器，而是执行生命周期管理器。

可以拆成：

```
graph TD
    TE[Tool Executor]
    TE --- TL[Tool Lookup]
    TE --- IV[Input Validation]
    TE --- BV[Business Validation]
    TE --- PC[Permission Check]
    TE --- HA[Human Approval Hook]
```

- +-- Execution Context Injection
- +-- Timeout Control
- +-- Retry Policy
- +-- Cancellation
- +-- Idempotency
- +-- Logging / Tracing
- +-- Result Normalization
- +-- Error Observation
- +-- Runtime State Feedback

一句话：

Tool Executor
=
LLM Decision 到 External World 的受控执行边界

为什么 LLM 不能直接执行 Tool

这是理解 Agent Runtime 的关键分界线。

很多初学者容易把 Tool Calling 理解为：

用户
|
v
LLM
|
v
调用 API
|
v
返回结果

工业架构不是这样。

真实链路是：

Agent Runtime

User
|
v
Context Builder
|
v
LLM
|
| Tool Call Intent
v
Tool Executor
|
| validate
| permission
| execute
| retry
| timeout
| tracing
v
External World

LLM 永远只是：

Decision Maker

不是：

Execution Engine

原因一：安全边界

LLM 是概率系统。

用户说：

删除测试环境所有数据。

LLM 可能生成：

```
{
  "name": "delete_database",
  "arguments": {
    "environment": "test"
  }
}
```

但是 Runtime 必须继续判断：

当前用户是谁？
当前环境是否允许删除？
当前任务是否允许 destructive action？
是否需要 human approval？
是否有审计记录？

所以：

LLM 提议动作。
Runtime 决定是否执行。

原因二：可靠性

LLM 不负责：

timeout
retry
circuit breaker
cancellation
logging
tracing
idempotency

例如调用支付 API 超时：

payment_api timeout

LLM 不应该决定：

是否重试？
重试几次？
是否可能重复扣款？
是否应该降级？

这些属于 Executor。

原因三：业务系统不应该暴露给模型

内部订单服务可能是：

POST /internal/order/query/v3

参数可能包括：

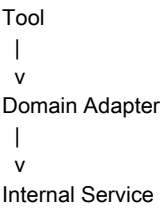
```
{
  "appId": "internal_app",
  "token": "...",
  "tenantId": "tenant_001"
}
```

LLM 不应该知道这些实现细节。

LLM 只应该看到：

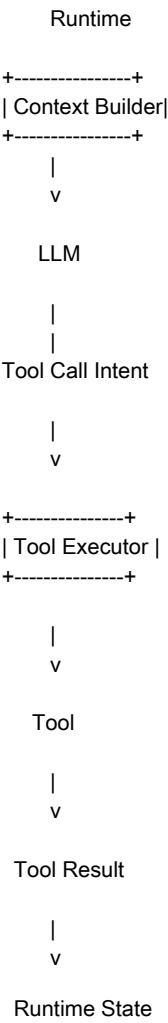
```
{
  "name": "query_order",
  "description": "查询订单状态",
  "parameters": {
    "type": "object",
    "properties": {
      "order_id": {
        "type": "string"
      }
    }
  }
}
```

Runtime 负责把抽象 Tool 转成具体业务调用：



Tool Executor 在 Runtime Loop 中的位置

完整 Agent Loop：



|
v

Next Agent Loop

Executor 位于：

Decision
|
v
Execution

之间。

它是：

从认知世界进入现实世界的边界层。

如果说 Context Builder 负责让 LLM 看见世界，那么 Tool Executor 负责让 Agent 改变世界。

Tool Call Intent 生命周期

LLM 输出的 Tool Call 不是执行结果。

它只是：

Intent

例如：

```
{  
  "name": "query_order",  
  "arguments": {  
    "order_id": "ORD-10001"  
  }  
}
```

完整生命周期如下。

Step 1 : LLM 生成 Intent

LLM
|
v
ToolCallIntent
{
 name,
 arguments
}

Step 2 : Executor 接收 Intent

Executor receives:

```
{  
  name: "query_order",  
  arguments: {  
    order_id: "ORD-10001"  
  }  
}
```

Step 3 : Registry 查找 Tool

Executor 通过 Tool name 查找 RuntimeTool：

```
const tool = registry.get(toolCall.name);
```


也就是：

```
registry.get("query_order");
```

找到：

```
{
  definition: {
    name: "query_order",
    inputSchema: ...
  },
  metadata: {
    riskLevel: "read-only"
  },
  execute(input, context) {
    ...
  }
}
```

Step 4：参数验证

Schema：

```
{
  "type": "object",
  "required": ["order_id"],
  "properties": {
    "order_id": {
      "type": "string"
    }
  }
}
```

LLM 输出：

```
{}
```

Executor 应该拒绝，不进入业务系统。

Step 5：权限检查

Tool：

```
refund_order
```

当前用户：

```
普通客服
```

需要权限：

```
refund:create
```

如果不满足：

```
reject or request approval
```

Step 6：执行 Tool Implementation

```
const rawResult = await tool.execute(toolCall.arguments, executionContext);
```

Step 7：结果标准化

成功：

```
{
  "ok": true,
  "output": {
    "status": "SHIPPED"
  }
}
```

```
}
```

失败：

```
{
  "ok": false,
  "error": {
    "code": "ORDER_NOT_FOUND",
    "message": "Order not found"
  }
}
```

Step 8：写回 Runtime State

Executor 不应该直接把原始结果塞给 LLM。

更准确：

```
Tool Result
|
v
Result Processor
|
v
Runtime State
|
v
Context Builder
|
v
LLM
```

Tool Result 在下一轮中表现为：

```
Observation
```

Intent、Registry、Router 与 Executor 的关系

一个重要问题是：

LLM 输出 Intent 后，从 Intent 到 Tool Registry 是怎样的？这个 Intent 是具体某个 Tool 吗？如果只是能力描述，从能力描述到 Registry 中找到 Tool 又是谁完成？

在现代 Tool Calling 中，LLM 输出的 Intent 通常已经包含具体 Tool name，而不是模糊能力描述。

LLM 输出：

```
{
  "name": "query_logistics",
  "arguments": {
    "tracking_number": "SF123456"
  }
}
```

这里的：

```
name = query_logistics
```

已经是 Registry 中 Tool 的唯一标识。

所以执行阶段不是语义搜索：

根据能力描述找 Tool

而是精确查找：

```
registry.get(toolCall.name)
```

完整链路：

```
Tool Registry
|
| 保存全部能力
v
Tool Router
|
| 缩小候选能力
v
Context Builder
|
| 暴露 Tool Schema
v
LLM
|
| 选择具体 Tool name
v
Tool Call Intent
|
| name = query_logistics
v
Tool Executor
|
| registry.get(name)
v
Tool Implementation
```

因此：

Router 是 LLM 推理前的候选集筛选。
Executor 是 LLM 推理后的执行调度。

Tool Schema 是 LLM 行动空间的边界

LLM 为什么能输出具体 Tool name？

因为 Runtime 在每次调用 LLM 前，会把当前可用的 Tool Definition 注入到请求中。

注意，不一定是在 Agent 初始化时一次性把全部 Tool 注入给 LLM。

更准确：

Agent Runtime 初始化：
Tool 注册进 Registry

每一轮 LLM 调用前：
Runtime 根据任务、权限、状态、环境筛选 Tool
Context Builder 把候选 Tool Schema 注入给 LLM

LLM 推理：
从当前可见 Tool 中选择具体 Tool name

Executor 执行：
通过 Tool name 找到真实实现

也就是：

```
Register once
|
v
Select every turn
|
v
LLM chooses
|
```

v
Executor executes

如果没有注入 Tool Schema，LLM 根本不知道可以调用哪个动作。

例如只给 LLM：

我的订单什么时候到？

没有 tools。

LLM 可能回答：

请提供订单号，我帮你查询。

或者更糟糕：

您的订单预计明天送达。

因为它不知道 Runtime 存在：

query_logistics

这个能力。

所以：

Tool Schema 是 LLM 行动空间的边界。

Tool Executor 的职责边界

Tool Executor 的职责不是：

替 LLM 思考

也不是：

管理全部 Tool

更不是：

编排完整业务流程

它的核心职责是：

可靠执行一个已经被选中的 Tool Call Intent

可以这样分层：

LLM Decision Layer
|
| 选择动作
v
Tool Executor
|
| 执行动作
v
Tool Implementation
|
| 调业务系统
v
External World

更完整：

Tool Executor
|
+-- 不负责选择 Tool
+-- 不负责暴露 Tool Schema

+-- 不负责长期记忆
+-- 不负责多步骤 Workflow 编排
|
+-- 负责单次 Tool Call 的受控执行

Tool Execution Context

普通函数可能是：

```
queryOrder(orderId);
```

Agent Tool 更像：

```
execute(input, executionContext);
```

因为 Tool 执行需要知道运行现场。

一个最小 ToolExecutionContext：

```
type ToolExecutionContext = {  
  userId: string;  
  sessionId: string;  
  workspace?: string;  
  permissions: string[];  
  runtimeState: RuntimeState;  
  logger: Logger;  
  traceId: string;  
  abortSignal: AbortSignal;  
};
```

这些字段不是 LLM 协议标准。

需要区分：

LLM Protocol

|
| Tool name
| arguments
| tool result
v

模型接口层

Runtime Internal Contract

|
| userId
| permissions
| workspace
| logger
| traceId
v

框架 / 企业内部设计

不同 Runtime 可以有不同 Context：

OpenAI Agents SDK:

context / session / handoff / guardrail

Claude Code:

workspace / filesystem / terminal / permission

LangGraph:

state / node / edge / checkpoint

企业 Agent:

user / tenant / role / approval / audit

行业更统一的是：

- Tool name
- Tool schema
- Arguments
- Tool result

不是：

- permissions 字段必须叫什么
- workspace 字段必须叫什么
- logger 字段必须叫什么

这和传统后端类似：

HTTP 标准化了 Method、Headers、Body。
但 req.user、req.tenant、req.permissions 是框架或公司自己定义的。

Input Validation

为什么 Executor 还需要参数校验？

因为 Tool Schema 虽然给了 LLM，但 LLM 输出不是可信输入。

Schema：

```
{
  "type": "object",
  "properties": {
    "order_id": {
      "type": "string"
    }
  },
  "required": ["order_id"]
}
```

LLM 可能输出：

```
{
  "order_id": 12345
}
```

也可能输出：

```
{
  "order_id": "DROP TABLE orders"
}
```

所以 Executor 至少需要两层验证。

第一层：Schema Validation

检查格式：

- required fields
- type
- enum
- array length
- object shape

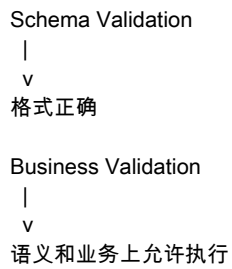
第二层：Business Validation

检查业务约束：

- 退款金额是否超过阈值？
- 文件路径是否在 workspace 内？
- 命令是否包含危险操作？

目标订单是否属于当前 tenant ?
当前状态是否允许执行这个 action ?

因此：



Permission Check 与 Human Approval 的边界

Part E 需要引出 Permission，但不展开全部细节，因为 Part F 会专门学习 Permission & Human Approval。

Executor 是 Permission 的执行入口。

它要在真正执行前调用：

Permission Layer

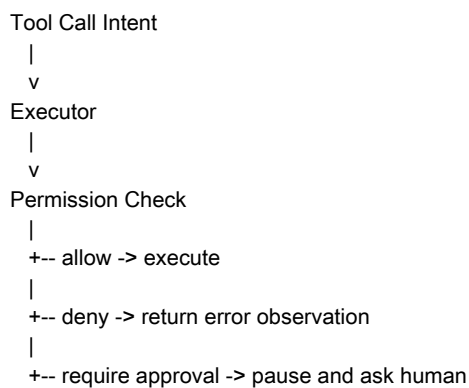
例如：

Tool Call:
delete_file

Risk:
destructive

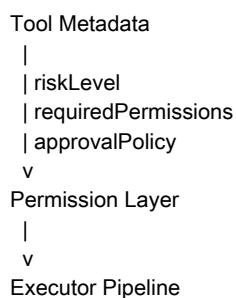
Policy:
workspace:write required
human approval required for deleting more than 10 files

执行链路：



Executor 不应该把权限逻辑散落到每个 Tool 里。

更好的结构是：



Part F 会继续回答：

Agent 已经拥有执行能力后，Runtime 如何限制它的行为？

Reliability：Timeout、Retry、Cancellation

真实世界的 Tool 调用会失败。

例如：

```
search_web
payment_api
execute_command
browser_task
database_query
```

可能遇到：

```
network timeout
rate limit
service unavailable
partial failure
user cancellation
```

Executor 需要提供可靠性机制。

Timeout

每个 Tool 应该有执行时间上限：

```
metadata: {
  timeoutMs: 30_000
}
```

避免 Agent 卡死。

Retry

某些失败可以重试：

```
ECONNRESET
HTTP 503
temporary rate limit
```

某些失败不能重试：

```
payment submitted
refund created
email sent
delete executed
```

因此 Retry Policy 必须和 Tool 的语义绑定：

```
read-only Tool:
  retry is usually safe
```

```
write Tool:
  retry must consider idempotency
```

```
external-send Tool:
  retry may duplicate side effects
```

Cancellation

长时间任务需要支持取消：

```
abortSignal: AbortSignal
```


例如用户说：

停一下，不要继续跑命令了。

Executor 应该把取消信号传给 Tool Implementation。

Idempotency

幂等是 Agent Tool Executor 的重要工业能力。

例如用户说：

给客户退款。

Tool：

refund_order

第一次调用成功，但网络断开。

Runtime 不知道是否成功，于是重试：

refund_order

结果可能：

退款两次

所以 Executor 需要 Idempotency Key：

```
{
  "tool": "refund_order",
  "request_id": "exec_abc123"
}
```

业务系统看到：

exec_abc123 已处理

直接返回之前结果。

Tool Metadata 可以标记：

```
type ToolMetadata = {
  idempotency: "required" | "supported" | "not_applicable";
};
```

经验规则：

Read Tool:
usually idempotent

Write Tool:
must reason carefully

Payment / Refund / Email:
idempotency is critical

Error Observation

Agent 与传统程序的最大区别之一是：

错误不一定终止任务。
错误可以成为下一轮推理的 Observation。

传统程序：

Error
|
v
Exception
|
v
End

Agent :

Tool Error
|
v
Observation
|
v
LLM
|
v
Re-plan

例如 Tool 返回 :

```
{
  "ok": false,
  "error": {
    "code": "ORDER_NOT_FOUND",
    "message": "Order ORD-10001 was not found"
  }
}
```

Runtime 可以转成 :

Observation:
订单 ORD-10001 不存在, 请确认订单号。

LLM 下一轮可能 :

请求用户确认订单号
尝试用手机号查询订单
结束任务并说明原因

所以 :

Tool Error = Environment Feedback

在 ReAct 语境下, 它就是 :

Observation

Tool Result Processing

Tool Result 不应该总是直接返回给 LLM。

简单 Demo :

Tool
|
v
Result
|
v
LLM

工业 Runtime :

Tool
|

```
v
Raw Result
|
v
Result Processor
|
v
Runtime State
|
v
Context Builder
|
v
LLM-visible Observation
```

原因是 Tool 结果可能：

- 太大
- 包含敏感信息
- 包含模型不需要的字段
- 需要脱敏
- 需要摘要
- 需要结构化成 Observation
- 需要只保存到 State，不进入本轮 Context

例如数据库返回：

```
{
  "orders": [
    "... 10000 rows ..."
  ]
}
```

不能直接塞进 Context。

Result Processor 需要做：

- filter
- summarize
- redact
- normalize
- persist
- project

这就是 Day04 和 Day05 的连接点：

- Tool Result 是 Runtime State 的一部分。
- 是否进入 LLM Context，由 Context Builder 决定。

Streaming 与 Long-running Tool

很多 Tool 不是 100ms 返回。

例如：

- execute_command
- run_analysis
- browser_task
- deploy_service
- scan_repository
- generate_report

可能耗时：

- 5 分钟
- 30 分钟
- 更久

简单：

```
await tool.execute();
```

不够。

工业设计需要支持：

Async Job Execution
Streaming Event
Progress Observation
Cancellation
Checkpoint

一种结构：

```
Tool Executor
|
v
start job
|
v
return task_id
|
v
background execution
|
v
streaming events
|
v
Runtime State update
```

例如：

```
{
  "task_id": "task_001",
  "status": "running"
}
```

然后不断产生：

Observation Stream:

Step 1: 打开网页
Step 2: 搜索订单
Step 3: 提取结果
Step 4: 完成

后续 Day07 Streaming Event、Day08 Human Approval、Day05 Part H Multi Tool Loop
都会继续连接这一点。

Executor 与 Workflow 的边界

Executor 负责：

怎么执行一个动作

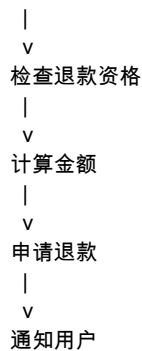
Workflow 负责：

多个动作如何组合

不要把完整业务流程塞进一个 Tool。

例如退款流程：

查询订单



不建议写成：

```
refundTool.execute(async () => {
  await queryOrder();
  await checkRefund();
  await refund();
  await sendEmail();
});
```

这样 Tool 变成 Workflow。

更好的结构：

```
Workflow
|
+-- query_order Tool
|
+-- check_refund Tool
|
+-- refund_order Tool
|
+-- send_email Tool
```

职责分离：

Executor:
执行单个动作

Workflow:
编排多个动作

Static Tool 与 Dynamic Tool

用户提出了一个很重要的类比：

前端搭建体系中，会把物料放到远端 CDN 上，SSR 动态拉取包配置，动态拼接包。一个好的 Agent 系统是不是也会动态 Tool，防止每次更改 Tool 都要发版？

答案是：

是。工业级 Agent 系统通常会从静态 Tool 列表演进到 Dynamic Tool Discovery / Dynamic Tool Registry。

但不意味着所有 Tool 都应该动态。

Static Tool

适合：

- 核心业务能力
- 高风险能力
- 强稳定性能力
- 强权限能力
- Runtime 基础能力

例如：

```
query_order
refund_order
execute_shell
write_file
delete_file
```

这些更适合代码内置、严格审核、随 Runtime 发版。

Dynamic Tool

适合：

```
外部生态能力
SaaS 集成
插件能力
MCP Server 提供的能力
租户差异化能力
环境发现能力
```

例如：

```
github.create_issue
slack.send_message
notion.search_page
jira.create_ticket
google_drive.read_file
```

最佳实践通常是：

```
Core Capability
|
v
Static Built-in Tool

External / Extension Capability
|
v
Dynamic Tool / MCP / Plugin
```

MCP 与 Dynamic Tool Registry

MCP 可以理解为：

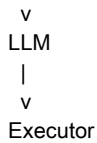
让外部能力以标准方式进入 Agent Runtime 的协议。

MCP Server 提供：

```
search_repo
create_issue
get_pull_request
```

Runtime 连接后：

```
MCP Server
|
v
Tool Discovery
|
v
Tool Registry
|
v
Context Builder
|
```



LLM 不关心 Tool 来自哪里。

它只看到：

```
{
  "name": "github.create_issue",
  "description": "Create a GitHub issue",
  "parameters": {}
}
```

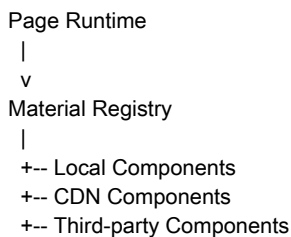
真正执行时，Executor 根据 Tool Implementation 或 Tool Adapter 再调用 MCP Server。

因此：

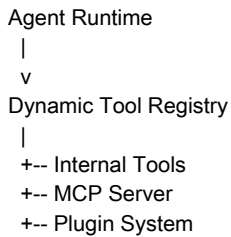
- MCP 不替代 Tool Registry。
- MCP 是外部 Tool 进入 Registry 的来源之一。

前端动态物料体系类比 Agent Tool Platform

前端搭建平台：



Agent Runtime：



对应关系：

| 前端搭建体系 | Agent Runtime | -|-| | 组件 | Tool | | 组件协议 | Tool Schema | | 物料中心 | Tool Registry | |
CDN | Tool Server / MCP Server | | 动态加载 | Tool Discovery | | 页面 Runtime | Agent Runtime | | 页面配置 |
Agent Context | | 渲染器 | Executor |

更高级的理解：

传统前端 Runtime
=
Runtime + Dynamic Component + Configuration

Agent Runtime
=
Runtime + Dynamic Capability + LLM Decision

所以 Agent Runtime 可以被理解为：

一个能够动态加载能力，并让 LLM 在这些能力中进行决策的智能运行时。

mini-agent-runtime 中的最小数据模型

Part E 可以先定义最小执行模型。

ToolCallIntent

```
export type ToolCallIntent = {  
  id: string;  
  name: string;  
  arguments: unknown;  
};
```

ToolDefinition

```
export type ToolDefinition = {  
  name: string;  
  description: string;  
  inputSchema: JSONSchema;  
  outputSchema?: JSONSchema;  
  errorSchema?: JSONSchema;  
};
```

ToolMetadata

```
export type ToolMetadata = {  
  source: "builtin" | "mcp" | "plugin" | "remote";  
  riskLevel: "read" | "write" | "destructive" | "external_send";  
  requiredPermissions?: string[];  
  timeoutMs?: number;  
  retryPolicy?: RetryPolicy;  
  idempotency?: "required" | "supported" | "not_applicable";  
};
```

RuntimeTool

```
export type RuntimeTool = {  
  definition: ToolDefinition;  
  metadata: ToolMetadata;  
  execute: (  
    input: unknown,  
    context: ToolExecutionContext  
  ) => Promise<ToolExecutionRawResult>;  
};
```

ToolExecutionContext

```
export type ToolExecutionContext = {  
  userId: string;  
  sessionId: string;  
  workspace?: string;  
  permissions: string[];  
  runtimeState: RuntimeState;  
  logger: Logger;  
  traceId: string;  
  abortSignal: AbortSignal;  
};
```

ToolExecutionResult

```
export type ToolExecutionResult =  
| {  
  ok: true;  
  toolCallId: string;  
  output: unknown;  
  observation: string;  
}
```



```

    }
  | {
    ok: false;
    toolCallId: string;
    error: {
      code: string;
      message: string;
      retryable?: boolean;
    };
    observation: string;
  };
};

```

mini-agent-runtime 最小实现草图

最小 Executor :

```

export class ToolExecutor {
  constructor(
    private readonly registry: ToolRegistry,
    private readonly validator: ToolInputValidator,
    private readonly permission: PermissionService,
    private readonly resultProcessor: ToolResultProcessor
  ) {}

  async execute(
    toolCall: ToolCallIntent,
    context: ToolExecutionContext
  ): Promise<ToolExecutionResult> {
    const tool = this.registry.get(toolCall.name);

    if (!tool) {
      return {
        ok: false,
        toolCallId: toolCall.id,
        error: {
          code: "TOOL_NOT_FOUND",
          message: `Tool ${toolCall.name} is not registered.`,
          retryable: false,
        },
        observation: `Tool ${toolCall.name} is not available.`,
      };
    }

    const validation = this.validator.validate(
      toolCall.arguments,
      tool.definition.inputSchema
    );

    if (!validation.ok) {
      return {
        ok: false,
        toolCallId: toolCall.id,
        error: {
          code: "INVALID_TOOL_ARGUMENTS",
          message: validation.message,
          retryable: false,
        },
        observation: `Tool ${toolCall.name} could not run because its arguments were invalid: ${validation.message}`,
      };
    }

    const decision = await this.permission.check(tool, context);

    if (decision.type === "deny") {
      return {

```

```

    ok: false,
    toolCallId: toolCall.id,
    error: {
      code: "PERMISSION_DENIED",
      message: decision.reason,
      retryable: false,
    },
    observation: `Tool ${toolCall.name} was denied by runtime policy: ${decision.reason}`,
  };
}

if (decision.type === "approval_required") {
  return {
    ok: false,
    toolCallId: toolCall.id,
    error: {
      code: "APPROVAL_REQUIRED",
      message: decision.reason,
      retryable: true,
    },
    observation: `Tool ${toolCall.name} requires human approval before execution.`,
  };
}

try {
  const rawResult = await tool.execute(toolCall.arguments, context);
  return this.resultProcessor.toExecutionResult(toolCall, rawResult);
} catch (error) {
  return this.resultProcessor.toErrorResult(toolCall, error);
}
}
}

```

这个实现还缺少：

```

timeout
retry
cancellation
idempotency
streaming
tracing

```

但它已经表达了 Part E 的最小核心：

```

lookup
validate
permission
execute
normalize
observe

```

工业术语映射

| 本节概念 | 工业术语 | |---| | Tool Executor | Execution Engine / Execution Runtime | | Tool Call Intent | Action Request | | Tool Registry Lookup | Service Discovery / Capability Lookup | | Tool Definition | Action Contract | | Input Validation | Input Guard | | Business Validation | Domain Guard | | Permission Check | Policy Enforcement Point | | Human Approval | Human-in-the-loop | | Retry | Reliability Middleware | | Timeout | Execution Control | | Cancellation | Abort Signal / Task Cancellation | | Idempotency | Distributed System Safety | | Tool Result Processor | Observation Processor | | Error Observation | Environment Feedback | | Long-running Tool | Async Job Execution | | Streaming Tool | Event-driven Execution | | Dynamic Tool Registry | Dynamic Capability Platform | | MCP Tool | Remote Capability Provider |

面试视角

Q1：为什么 Tool Executor 必须存在？

因为 LLM 只能产生 Tool Call Intent，不能直接控制真实系统。Executor 负责参数校验、权限控制、可靠执行、错误处理、结果标准化和审计追踪。

Q2：Tool Executor 和 Tool Registry 有什么区别？

Registry 管理有哪些能力，Executor 管理如何执行能力。

Registry:
Capability Management

Executor:
Runtime Managed Execution

Q3：LLM 输出的 Intent 是能力描述还是具体 Tool？

通常是具体 Tool Call，包含 Tool name 和 arguments。Tool name 来自 Runtime 在本轮请求中注入的候选 Tool Schema。Executor 通过 registry.get(name) 找到 Tool Implementation。

Q4：ToolExecutionContext 是模型协议标准吗？

不是。Tool name、arguments、schema、result
更接近模型接口协议；userId、permissions、workspace、logger、traceId 这些属于 Runtime 内部契约，由框架或企业自己设计。

Q5：Tool Error 为什么可以继续 Agent Loop？

因为 Agent 可以把错误视为 Observation，而不是终止异常。LLM 能根据错误反馈重新规划、补充信息或请求用户确认。

Q6：Tool Executor 和普通函数调用有什么区别？

普通函数调用：

input -> function -> output

Agent Tool Executor：

```
Intent
|
v
Validation
|
v
Policy
|
v
Execution
|
v
Observation
|
v
Next Reasoning
```

Q7：为什么 Executor 不应该包含完整业务流程？

因为 Executor 管单个动作的可靠执行，Workflow 管多个动作的组合编排。如果把流程塞进 Tool，Tool 会变成 Workflow，能力粒度和职责边界都会混乱。

下一节学习计划

下一节进入：

Day05 Part F：Permission & Human Approval

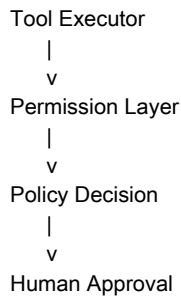
Part E 解决：

Runtime 如何执行动作？

Part F 解决：

当 Agent 已经可以调用 Tool 并影响真实世界时，Runtime 如何决定这个动作是否允许执行？

学习链路：



重点：

RBAC 在 Agent 中的应用

Policy Engine

Action Guard

Human-in-the-loop

高风险操作审批

Permission 与 Tool Metadata 的关系

Approval 如何暂停和恢复 Agent Loop

工业映射：

Agent Permission
~=
企业权限系统 + 审批流

写书 TODO

未来整理《Agent Runtime 工程设计》时，可以新增章节：

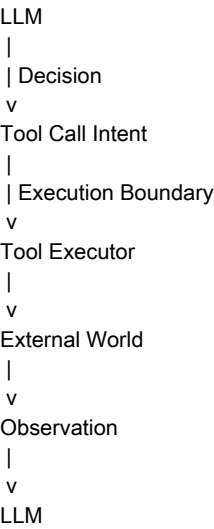
第 X 章：Tool Execution Engine

章节结构：

1. Tool Calling 不等于 Tool Execution
2. Intent 与 Execution 分离
3. Tool Executor 为什么是 Runtime 执行内核
4. Executor Middleware Pipeline
5. Tool Execution Context
6. Input Validation 与 Business Guard
7. Permission、Approval 与 Executor 的衔接
8. Tool Result Processing 与 Observation
9. Reliability Engineering：Timeout、Retry、Cancellation、Idempotency
10. Streaming 与 Long-running Tool

- 11. Executor 与 Workflow 的职责边界
- 12. Agent Executor 与传统系统执行框架对比

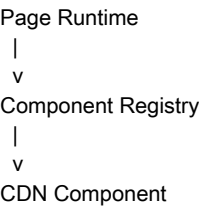
新增核心图：



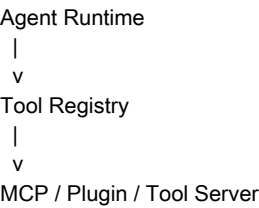
写书素材

素材一：前端动态物料体系类比 Agent Dynamic Tool System

传统前端：



Agent：

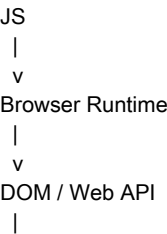


这个素材适合说明：

Agent Platform Engineering
=
Runtime + Dynamic Capability + Governance

素材二：Agent Runtime = 浏览器 Runtime

浏览器：



v
OS

Agent :

LLM
|
v
Agent Runtime
|
v
Tool Executor
|
v
External System

素材三：Tool Executor = Agent 世界的 Kernel

OS :

Application
|
v
System Call
|
v
Kernel
|
v
Hardware

Agent :

LLM
|
v
Tool Call
|
v
Executor
|
v
External World

Executor 是 Agent 接触真实世界的唯一受控入口。

本 Part 核心认知升级

Part E 完成后，认知需要从：

Tool Executor = 调用一个函数

升级为：

Tool Executor
=
Agent Execution Kernel
+
安全边界
+
可靠性控制
+
现实世界接口
+
Observation 生成器

更完整：

Agent Action
=
LLM Decision
+
Tool Registry
+
Tool Executor
+
Permission
+
Observation Feedback

Agent Runtime 的执行系统不是让 LLM 直接操作世界，而是：

Runtime 给 LLM 一个受控行动菜单。
LLM 从菜单中选择动作。
Executor 像厨房一样执行动作。
Result Processor 把结果变成 Observation。
Context Builder 再把 Observation 投影给 LLM。

这里形成 Day05 到目前为止的完整链路：

```
graph TD
    TS[Tool Schema] --> VR[ ]
    VR --> TR[Tool Registry]
    TR --> VR2[ ]
    VR2 --> TRouter[Tool Router]
    TRouter --> VR3[ ]
    VR3 --> LLM[LLM Tool Decision]
    LLM --> VR4[ ]
    VR4 --> TE[Tool Executor]
    TE --> VR5[ ]
    VR5 --> P[Permission]
    P --> VR6[ ]
    VR6 --> O[Observation]
    O --> VR7[ ]
    VR7 --> RS[Runtime State]
    RS --> VR8[ ]
    VR8 --> NAL[Next Agent Loop]
```

本章思考题

为什么 LLM 不能直接调用数据库或内部业务 API？

Tool Call Intent 和 Tool Execution 有什么区别？

LLM 为什么能输出具体 Tool name？

Tool Router 和 Tool Executor 分别发生在 LLM 调用前还是调用后？

ToolExecutionContext 为什么不属于模型协议标准？

Schema Validation 为什么不能完全相信 LLM？

Business Validation 和 Schema Validation 的区别是什么？

Retry 应该属于 Tool Implementation 还是 Tool Executor ?

对退款、发邮件、支付这类 Tool , 为什么 Idempotency 很关键 ?

Tool Error 为什么是 Observation , 而不只是 Exception ?

Tool Result 为什么不应该总是直接返回给 LLM ?

Long-running Tool 应该如何设计 ?

Executor 和 Workflow 的职责边界在哪里 ?

一个 Agent 系统中哪些 Tool 适合静态内置, 哪些适合动态发现 ?

MCP Server 提供的能力为什么最终仍要进入 Runtime Registry ?

Tool Executor 更像 API Gateway、Job Executor , 还是 OS Kernel ? 为什么 ?

前置问题回收

本节回收了几个之前挂起的问题。

问题一 : Agent 本质是不是软件工程 ?

Part E 基本验证 :

是, 但不止是。

Tool Executor 里大量概念来自传统软件工程 :

- Validation
- Permission
- Retry
- Timeout
- Logging
- Tracing
- Idempotency
- Error Handling

Agent 真正新增的是 :

LLM Decision Layer

也就是 :

过去由程序员硬编码的 Action Selection ,
现在部分交给 LLM 根据目标和上下文动态完成。

问题二 : Tool 是否应该动态加载 ?

Part E 给出初步答案 :

核心能力静态内置。
扩展能力动态发现。
外部生态能力通过 MCP / Plugin / Tool Server 接入。

后续在 MCP、Plugin、Capability Platform 章节可以继续展开。

问题三 : Permission 在哪里接入 ?

Part E 只建立入口 :

Executor Pipeline 中的 Permission Check

Part F 将正式展开 :

Permission & Human Approval